

# API网关概要设计

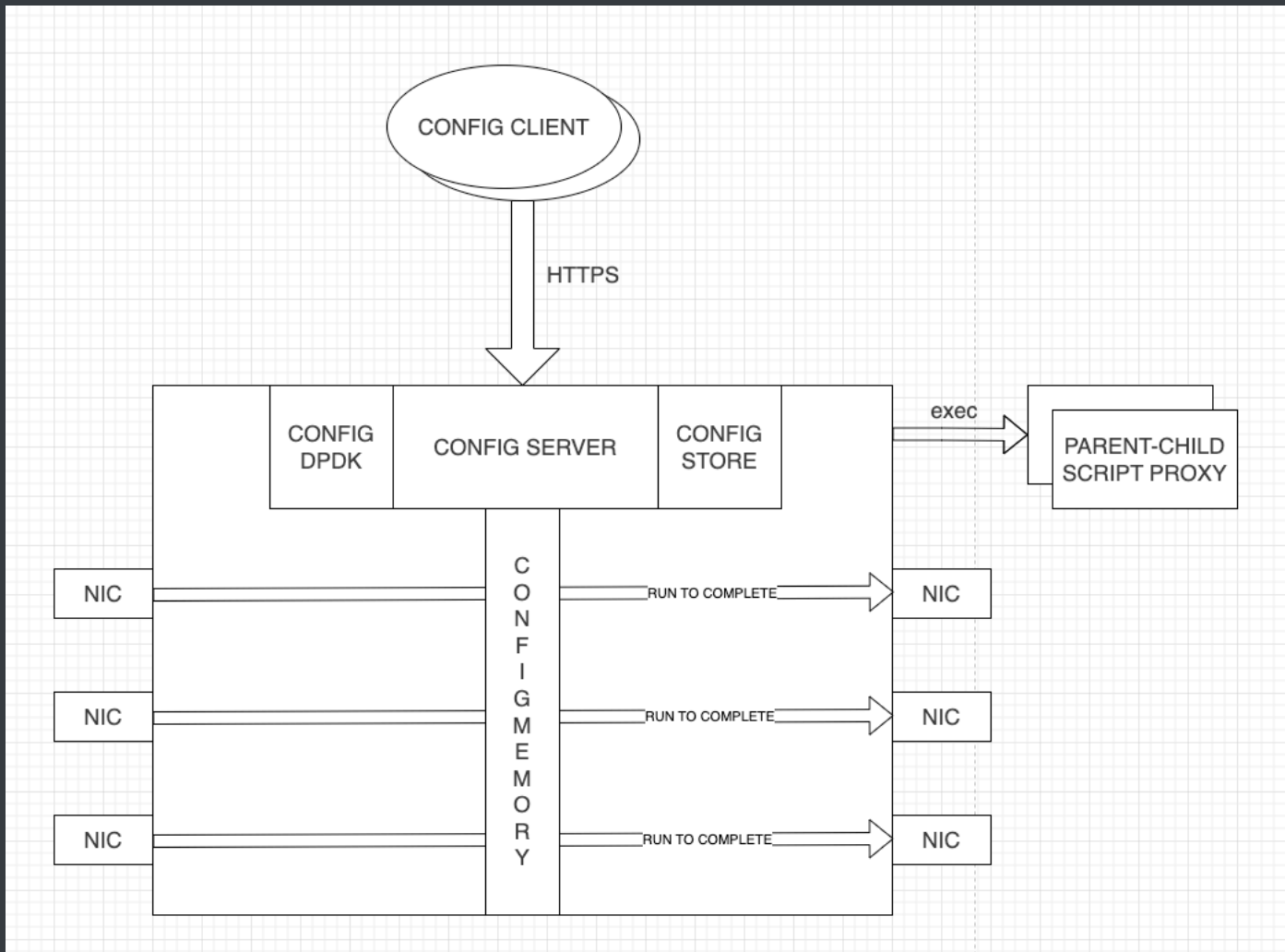
## API网关概要设计

- 一、API网关 - 配置驱动架构
- 二、CONFIG SERVER组件
  - 1、快速搭建HTTPS服务器
    - libmicrohttpd
    - Libevent + OpenSSL
    - Mongoose
    - Mongoose 示例
  - 2、YANG和DB
    - a、数据流示意图
    - b、libyang
      - i、demo
      - ii、YANG编码流程
      - iii、YANG语法
      - iv、工具
    - c、sysrepo
      - sysrepo四个数据库
      - 关键机制说明
      - 示例
      - 工具
  - 3、CONFIG DPDK
  - 4、父子进程脚本代理
- 三、APIGATEWAY
  - 1、main
  - 2、API线程
  - 3、DPDK线程
  - 4、网卡队列内存注册
  - 5、会话表设计

## 一、API网关 – 配置驱动架构

## 单机API网关配置驱动架构

用户在CONFIG CLIENT生成配置，通过HTTPS把配置下发给CONFIG SERVER，CONFIG SERVER存储到CONFIG STORE，并作用于PARENT-CHILD SCRIPT PROXY、CONFIG MEMORY和CONFIG DPDK(CONFIG DPDK分为启动前和启动后CONFIG CLIENT配置)，通过PARENT-CHILD SCRIPT PROXY作用于内核，通过CONFIG MEMORY和CONFIG DPDK作用于数据包，构成配置驱动架构。如下所示：



## 二、CONFIG SERVER组件

### 1、快速搭建HTTPS服务器

libmicrohttpd

优点：

- 轻量级：纯 C 实现，二进制仅约 32KB（无 SSL 支持），适合嵌入式系统

- **简单 API**: 提供基础的 HTTP 1.1 功能, 支持多线程模型 (select/poll/epoll)
- **低资源占用**: 适合资源受限环境, 如 IoT 设备

缺点:

- **功能有限**: 仅支持基础 HTTP/HTTPS, 高级功能需自行实现(不支持分块传输)
- **开发复杂度**: 处理 POST 数据或 JSON 需手动解析, 无内置高级工具
- **性能瓶颈**: 高并发场景下不如事件驱动库 (如 Libevent) 高效

适用场景:

- 嵌入式设备的小型 HTTP API 服务
- 快速搭建轻量级 HTTPS 服务器 (需搭配 OpenSSL)

## Libevent + OpenSSL

优点:

- **高性能**: 事件驱动模型 (epoll/kqueue) 支持高并发, 适合大规模连接
- **灵活性**: 可深度定制协议 (如自定义 HTTPS 逻辑), 适合复杂需求
- **成熟稳定**: Libevent 是业界标准事件库, OpenSSL 提供完整 TLS 支持

缺点:

- **开发复杂度高**: 需手动处理 SSL/TLS 握手、证书管理等, 代码量较大
- **依赖组合**: 需同时集成 Libevent 和 OpenSSL, 增加部署复杂度

适用场景:

- 高性能 HTTPS 代理或网关
- 需要自定义协议或非阻塞 I/O 的复杂应用

## Mongoose

优点:

- **极简设计**：仅需 `mongoose.c` 和 `mongoose.h`，无外部依赖
- **事件驱动**：非阻塞 API，资源占用低，适合嵌入式设备
- **多协议支持**：内置 HTTP/HTTPS

有大量知名企业使用Mongoose作为设备Web服务端，如NASA在国际空间站中部署Mongoose，用于遥测数据交换和远程控制，西门子在工业自动化设备中嵌入Mongoose，提供Web UI和RESTful API服务，谷歌在部分物联网设备使用Mongoose作为轻量级Web服务器，苹果在iPhone应用中嵌入Mongoose实现本地HTTP服务等。

缺点：

- **性能中等**：单线程事件循环，高并发时需优化（如线程池）
- **功能简化**：高级HTTP功能（如流式上传）需自行实现

适用场景：

- 嵌入设备的WEB控制界面
- 需要轻量级服务的场景

## Mongoose 示例

只需要引入`mongoose.c`和`mongoose.h`即可，要支持HTTPS，编译时带入宏：`-DMG_TLS=MG_TLS_BUILTIN`或者`-DMG_TLS=MG_TLS_OPENSSL`，`openssl`需要依赖`ssl`和`lcrypto`库，HTTPS服务端示例如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <jansson.h>

#include "check.h"
#include "mongoose.h"

#define JSON_FORMAT "Content-Type: application/json\r\n"
```



```

"MHcCAQEEIAVdo8UAScxG7jiuNY2UZESNX/KPH8qJ0u0gOMMsAzYWoAoGCCqGSM49\n"

"AwEHoUQDQgAEqN6BIhvgbk7ecmUcn8Da9Avkj/uDNERtqWJG9r/or26X4u9jR5Jl\n"
"4hf5Gx17YJkq5/z3k6ogPDPpoAYWIw1/sw==\n"
"-----END EC PRIVATE KEY-----\n";
#endif // 0

static void cb(struct mg_connection *c, int ev, void *ev_data)
{
    int ret = 0;
    const char *err_msg = NULL;
    static __thread char *content = NULL;
    static __thread const char *err_404_msg = "
{\"code\": \"1\", \"message\": \"Endpoint not found\"}";

    switch (ev) {
#if 0
    case MG_EV_ACCEPT:
        if (c->fn_data != NULL) {
            struct mg_tls_opts opts = {0};
            opts.ca = mg_str(s_tls_ca);
            opts.cert = mg_str(s_tls_cert);
            opts.key = mg_str(s_tls_key);
            mg_tls_init(c, &opts);
        }

        break;
#endif // 0
    case MG_EV_HTTP_MSG:
        struct mg_http_message *msg = ev_data;

        switch (msg->method.len) {
        case 3:
            if (strncasecmp(msg->method.buf, "GET", 3) == 0) {

```

```

        mg_http_reply(c, 200, JSON_FORMAT, "GET: %.*s\n", msg-
>uri.len, msg->uri.buf);
    } else if (strncasecmp(msg->method.buf, "PUT", 3) == 0) {
        mg_http_reply(c, 200, JSON_FORMAT, "PUT: %.*s\n", msg-
>uri.len, msg->uri.buf);
    } else {
        goto _quit;
    }

    break;
case 4:
    if (strncasecmp(msg->method.buf, "POST", 4) != 0) {
        goto _quit;
    }

    ret = check(msg->body.buf, msg->uri.buf, msg->uri.len,
&err_msg);
    if (ret != 0) {
        mg_http_reply(c, 501, "", "%s", err_msg);
        break;
    }

    mg_http_reply(c, 200, JSON_FORMAT, "POST: %d: %s\n", msg-
>body.len, msg->body.buf);
    break;
case 6:
    if (strncasecmp(msg->method.buf, "DELETE", 6) != 0) {
        goto _quit;
    }

    mg_http_reply(c, 200, JSON_FORMAT, "DELETE Hello World!:
%.*s\n", msg->uri.len, msg->uri.buf);

    break;
default: _quit:

```

```

        mg_http_reply(c, 404, JSON_FORMAT, err_404_msg);
        break;
    }

    break;
default:
    break;
}
}

int main(int argc, char *argv[])
{
    struct mg_mgr mgr;
    struct mg_connection *c;
    const char *http_url = "http://0.0.0.0:1080";

    mg_log_set(MG_LL_INFO);
    mg_mgr_init(&mgr);

    if ((c = mg_http_listen(&mgr, http_url, cb, (void *) 1)) == NULL)
    {
        fprintf(stderr, "Cannot to listen on %s.\n", http_url);
        exit(EXIT_FAILURE);
    }

    for (;;) {
        mg_mgr_poll(&mgr, 1000);
    }
    mg_mgr_free(&mgr);

    return EXIT_SUCCESS;
}

```

返回统一格式如下：



```

{
    "code" : 0,
    "data" : {
    }
}

{
    "code" : 1,
    "data" : {
        "errmsg" : "message"
    }
}

```

初期为了方便使用会使用HTTP，这里添加SSL比较容易，阿里云免费证书有三个月时效期，后期再加证书。

API实现风格会使用当前SMARTLB用的注册机制，如下：

```

#define API_POST(uri, func) \
    static __attribute__((constructor)) void func##_register(func, \
#uri) { \
        api_post_resgister(#uri, func); \
    } \
    static json_t *func(char *url, json_t *json) \
...

API_POST(/network/interface, network_interface_post)
{
    return 0;
}

// PUT和POST参数完全相同，且都要需要传递全量实例数据
// 前端支持PUT，在后端PUT就是POST，在后端实际的开发中，发现是PUT，则执行POST请求，故不需要单独实现
API_PUT(/network/interface, network_interface_put)

```

```

{
    return 0;
}

API_DELETE(/network/interface, network_interface_delete)
{
    return 0;
}

API_GET(/network/interface, network_interface_get)
{
    return 0;
}

```

## 2、YANG和DB

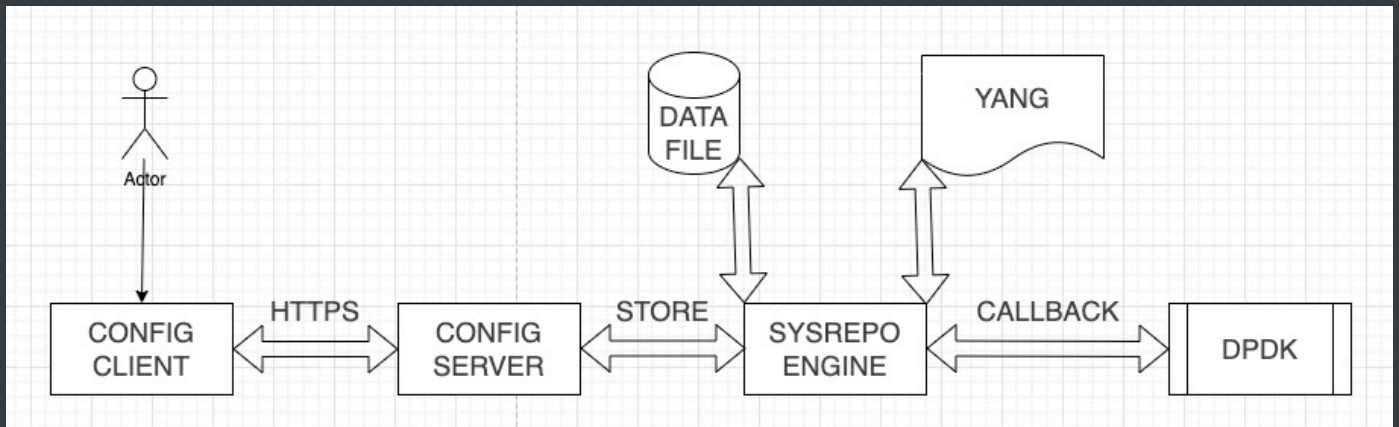
基于网络设备配置管理上的问题，IETF成立NETCONF工作组，发布了RFC4741-4744等协议来标准化网络设备的配置和管理，其中使用YANG语言建模配置数据，使用多配置数据库(running、startup)实现配置原子性。

根据NETCONF实现的Netopeer2依赖于libyang + sysrepo作为配置检查的scheme和配置存储库，libyang + sysrepo组合已经广泛应用在网络设备的配置中，我们也引入libyang和sysrepo作为API网关的YANG和DB。

**NOTE：**XPath(XML Path Language)是一种用于在XML文档中定位和选择节点的查询语言，主要用于精确操作YANG模型实例化的配置数据。有两个特性：

- 节点定位：通过路径表达式(/ietf-interfaces:interfaces/interface[name='eth0'])快速定位XML树中的特定节点
- 跨层级导航：使用绝对路径(/)、相对路径(//)、当前节点(.)和当前节点的父节点(..)遍历XML结构，适应复杂数据模型。

### a、数据流示意图



## b、libyang

libyang是一个YANG数据建模语言解析器和工具包。YANG是一种用于定义网络配置和操作数据的XML格式语言，广泛应用于网络设备的管理和设置。libyang提供了丰富的API，是的开发者能够轻松地解析、验证和操作YANG模型数据。该项目被广泛应用于libnetconf2、Netopeer2和sysrepo等项目中，是网络管理和配置领域的重要工具。

## i、demo

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <jansson.h>
#include <libyang/context.h>
#include <libyang/libyang.h>

static struct ly_ctx *s_ctx = NULL;

static void check_fini(void)
{
    ly_ctx_destroy(s_ctx);
}

static __attribute__((constructor)) void check_init(void)
{
    LY_ERR err = LY_SUCCESS;
    struct lys_module *module = NULL;
```

```

err = ly_ctx_new(NULL, 0, &s_ctx);
if (err != LY_SUCCESS) {
    fprintf(stderr, "ly_new_ctx failure.\n");
    exit(EXIT_FAILURE);
}

atexit(check_fini);

err = lys_parse_path(s_ctx, "yang/example.yang", LYS_IN_YANG,
&module);
if (err != LY_SUCCESS) {
    fprintf(stderr, "lys_parse_mem failure.\n");
    exit(EXIT_FAILURE);
}
}

int check(const char *json_str, const char **msg)
{
    LY_ERR err = LY_SUCCESS;
    struct lyd_node *node = NULL;

    err = lyd_parse_data_mem(s_ctx, json_str, LYD_JSON,
LYD_PARSE_ONLY, LYD_VALIDATE_MULTI_ERROR, &node);
    if (err != LY_SUCCESS) {
        *msg = ly_strerror(err);
        fprintf(stderr, "%s\n", ly_strerror(err));
        return !0;
    }

    err = lyd_validate_all(&node, s_ctx, LYD_VALIDATE_NO_STATE, NULL);
    if (err != LY_SUCCESS) {
        *msg = ly_strerror(err);
        fprintf(stderr, "%s\n", ly_strerror(err));
        return !0;
    }
}

```

```
    return 0;
}
```

## ii、YANG编码流程

YANG编码流程如下所示：

- 1、初始化句柄
- 2、加载和解析所有YANG模型文件
- 3、验证数据

## iii、YANG语法

YANG是一种数据建模语言，该语法依据[RFC7950](#)，用于定义网络设备的配置和状态数据结构，主要语法元素包括：

- 模块化
  - 模块(module)：组织模型的基本单元，包含module头，revision和definition等语句，可被其他模块引用或扩展。
  - 子模块(submodule)：是模块的一部分，用于拆分复杂模型，增强复杂性。子模块必须属于一个父模块。belongs-to指定父模块，通过include被父模块或其他子模块引用。
  - import：用于引用其他模块的定义，使当前模块可以复用外部模块的声明。必须指定prefix，用于区分不同模块的定义。
  - include：将子模块的内容合并到父模块中，实现代码的物理拆分。
  - grouping：定义可复用的数据块。仅在模块或子模块内有效，其他模块需通过import引入。定义数据结和逻辑块。
- 节点类型：
  - container：用于定义架构树中的内部数据节点，它有一个参数，它是一个标识符，后面跟着包含详细容器信息的子状态块。
  - leaf：叶子节点(如 leaf host-name {type string;})
  - leaf-list：多值节点(数组)

- list: 键值索引的列表
- choice: 用于定义互斥的节点分支, 确保数据模型中同一时刻只能存在一个有效的子节点。
- use: 用于引用其他模块或子模块中的定义
- typedef: 定义数据类型
- 约束:
  - key: 用于定义列表(list)的唯一性标识, 类似于数据库中的主键(PRIMARY KEY)。作为key的必须是叶子节点, 可用多个字段组合作为key, 用空格分隔"order-id product-id"。若未显示定义"mandatory true", key字段仍会强制非空。
  - unique: 用于确保数据节点值的唯一性, 与数据库中的UNIQUE约束类似, 它允许空值(null)且不强制作为主标识符。可用多个字段组合作为unique, 用空格风格。
  - config: 默认值true, 表示该节点是可配置的, 即可以通过网络管理协议进行修改, false表示该节点是只读的状态数据, 通常由设备自动生成或维护, 不可通过配置协议修改。
  - when: 定义条件约束。动态控制节点的可见性与有效性, 依赖其他节点的值。
  - must: 节点仅在表达式为真时有效。
  - mandatory: 强制节点必须存在。
  - min-elements和max-elements: 限制列表(list)或叶子列表(leaf-list)的元素数量。
  - range: 限制数值类型的取值范围。
  - length: 限制字符串或二进制类型的长度范围。
  - pattern: 正则表达式约束。限制字符串格式。
  - enumeration: 枚举约束。限制字段为预定义的枚举值。
  - default: 用于定义YANG模型中节点的默认值, 确保数据实例在未显式配置时自动填充默认值。仅对config true(配置节点)的节点有效, config false(状态节点)不支持默认值。
  - error-message: 为must约束定义自定义错误类型。
  - description: 用于为模块、节点或其他元素添加文本描述。
  - status: current - 节点处于活跃状态, deprecated - 节点已不推荐使用, obsolete - 节点已完全废弃。可以通过sr\_check\_compatibility()接口, sysrepoctl --check-compatibility检查YANG模型前后版本的兼容性。

- 注释：
  - 以 // 开头，注释内容到行尾结束
  - 以 /\* 开头，\*/ 结尾，可跨多行
- 示例：

```
module example {
    yang-version "1.1";
    namespace "example";
    prefix "example";

    import ietf-inet-types {
        prefix "inet";
    }

    revision "2025-05-16" {
        description "Initial version";
    }

    typedef identifier {
        type string {
            length '1..127';
            pattern '[_a-zA-Z][_0-9a-zA-Z]*';
        }
    }

    container system {
        list items {
            key "hostname";
            leaf hostname {
                type identifier;
                must "current() != 'invalid'";
                description "system hostname";
            }
            leaf-list ip-addresses {
                type inet:ip-address;
            }
        }
    }
}
```

```

        description "List of IP addresses.";
    }
}
}

# '{
#     "example:system" : {
#         "items" : [{
#             "hostname" : "ADC", # 不能为invalid
#             "ip-addresses" : [
#                 "192.168.10.1",
#                 "1.1.1.1"
#             ]
#         }]
#     }
# }'

```

#### iv、工具

介绍一个工具，编译libyang后自带的工具yanglint

检查yang模型语法是否正确：**yanglink interface.yang**

检查json数据是否匹配yang模型：**yanglint interface.yang interface.json**

yanglink功能强大，部分语法如下：

- -t 输入数据类型(yang/yin/xml/json)
- -f 输出格式(yang/yin/xml/json/tree/dot)
- -p 添加YANG模块搜索路径
- -o 将输出写入文件(如生成 tree 或 dot图)
- --strict 启动严格模式

#### c、sysrepo



sysrepo是一个基于YANG模型的配置和操作数据库，为应用程序提供统一的操作数据的接口。

sysrepo只是一个库(我们包含在DPDK中作为库部署，不单独部署为进程)，不是独立的进程，不会在后台时刻运行并消耗CPU、内存和磁盘资源等。

sysrepo采用订阅模式，配置变更时执行回调接口，回调接口中执行配置下发。如果回调接口中的配置下发失败，则配置变更会自动在sysrepo中被删除，如果回调接口返回成功，则配置被保存在sysrepo库中。保证了配置的一致性，即配置生效的配置一定在库中，未生效的配置一定不在库中(sysrepo采用了一致性的常用手段两阶段提交)。

## sysrepo四个数据库

- startup库：sysrepo中唯一的持久性数据配置库，它包含设备启动时的配置文件，系统启动后创建的第一个sysrepo连接时，会将配置文件从startup库copy到running库。
- running库：保存当前所运行时系统配置，当一个配置发生变化时并且设备需要重新配置时，running数据库需要修改。系统重启时不存在。
- candidate库：改库正常无效，实际使用需要镜像到running来生效。
- operational库：维护当前使用的配置，并且该库只可读。

## 关键机制说明

- 启动时从startup加载配置到running，配置在running变更后回写startup实现持久化；
- 对于配置存储到running事件进行持续监听，有变动则触发回调执行配置下发；
- 可扩展订阅任意数量的YANG模型，通过module\_name区分来源下发到不同的配置对象。

## 示例

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <jansson.h>
#include <sysrepo.h>
#include <libyang/context.h>
#include <libyang/libyang.h>
```

```

struct netconf {
    sr_conn_ctx_t *conn;
    sr_session_ctx_t *session;
    sr_subscription_ctx_t *subscript;
    struct private_data {
        const char *buf;
        size_t len;
    } pridata;
};

static struct netconf s_netconf;

static int check_config_load_cb(sr_session_ctx_t *session,
                                uint32_t sub_id,
                                const char *module_name,
                                const char *xpath,
                                sr_event_t event,
                                uint32_t operation_id,
                                void *private_data)
{
    struct private_data *data = private_data;

    printf("SR_EV_UPDATE = %d, SR_EV_CHANGE = %d, SR_EV_DONE = %d,
SR_EV_ABORT = %d, SR_EV_ENABLED = %d, SR_EV_RPC = %d, event = %d,
%.*s\n", SR_EV_UPDATE, SR_EV_CHANGE, SR_EV_DONE, SR_EV_ABORT,
SR_EV_ENABLED, SR_EV_RPC, event, (int)data->len, data->buf);

    // 匹配URL并执行下发
    // 这里采用的是两阶段提交，第一次是change事件，第二次是done事件，为了保证一致性。
    // 保存到startup数据库

    return 0;
}

```

```

static void check_fini(void)
{
    struct netconf *netconf = &s_netconf;

    if (netconf->subscript != NULL) {
        sr_unsubscribe(netconf->subscript);
        netconf->subscript = NULL;
    }

    if (netconf->session != NULL) {
        sr_session_stop(netconf->session);
        netconf->session = NULL;
    }

    if (netconf->conn != NULL) {
        sr_disconnect(netconf->conn);
        netconf->conn = NULL;
    }
}

static __attribute__((constructor)) void check_init(void)
{
    int ret = 0;
    LY_ERR err = LY_SUCCESS;
    struct lys_module *module = NULL;
    struct netconf *netconf = &s_netconf;
    const char *features[] = {NULL};

    ly_log_level(LY_LLDBG);
    sr_log_stderr(SR_LL_DBG);

    ret = sr_connect(SR_CONN_DEFAULT, &netconf->conn);
    if (ret != SR_ERR_OK) {
        fprintf(stderr, "Session error: %s\n", sr_strerror(ret));
        exit(EXIT_FAILURE);
    }
}

```

```
atexit(check_fini);

ret = sr_session_start(netconf->conn, SR_DS_RUNNING, &netconf->session);
if (ret != SR_ERR_OK) {
    fprintf(stderr, "session error: %s\n", sr_strerror(ret));
    exit(EXIT_FAILURE);
}

/*err = lys_parse_path(ctx, "yang/example.yang", LYS_IN_YANG,
&module);
if (err != LY_SUCCESS) {
    fprintf(stderr, "lys_parse_mem failure.\n");
    exit(EXIT_FAILURE);
}*/

ret = sr_install_module(netconf->conn,
                        "/home/liuwei/learn-
code/mongoose/yang/example.yang",
                        "/home/liuwei/learn-code/mongoose/yang/",
                        features);

if (ret != 0) {
    fprintf(stderr, "sr_install_module failure.\n");
    exit(EXIT_FAILURE);
}

ret = sr_module_change_subscribe(netconf->session, "example",
"/example:system", check_config_load_cb, &netconf->pridata, 0, 0,
&netconf->subscript);
if (ret != SR_ERR_OK) {
    fprintf(stderr, "subscribe error: %s\n", sr_strerror(ret));
    exit(EXIT_FAILURE);
}
}
```

```

int check(const char *json_str, const char *uri, size_t len, const
char **msg)
{
    int ret = 0;
    const struct ly_ctx *ctx = NULL;
    LY_ERR err = LY_SUCCESS;
    struct lyd_node *node = NULL;
    struct netconf *netconf = &s_netconf;

    ctx = sr_session_acquire_context(netconf->session);
    err = lyd_parse_data_mem(ctx, json_str, LYD_JSON, LYD_PARSE_ONLY,
LYD_VALIDATE_MULTI_ERROR, &node);
    if (err != LY_SUCCESS) {
        *msg = ly_strerror(err);
        lyd_free_tree(node);
        fprintf(stderr, "%s\n", ly_strerror(err));
        return !0;
    }

    err = lyd_validate_all(&node, ctx, LYD_VALIDATE_NO_STATE, NULL);
    if (err != LY_SUCCESS) {
        *msg = ly_strerror(err);
        lyd_free_tree(node);
        fprintf(stderr, "%s\n", ly_strerror(err));
        return !0;
    }

    ret = sr_edit_batch(netconf->session, node, "merge");
    if (ret != 0) {
        fprintf(stderr, "sr_edit_batch failure: %s\n",
sr_strerror(ret));
        lyd_free_tree(node);
        return !0;
    }
}

```

```

    /*ret = sr_set_item_str(netconf->session,
"/example:system/hostname", "host", NULL, 0);
    if (ret != 0) {
        fprintf(stderr, "sr_edit_batch failure: %s.\n",
sr_strerror(ret));
        return !0;
    }*/

    netconf->pridata.buf = uri;
    netconf->pridata.len = len;
    ret = sr_apply_changes(netconf->session, 30 * 1000);
    if (ret != 0) {
        fprintf(stderr, "load.\n");
        lyd_free_tree(node);
        return !0;
    }

    lyd_free_tree(node);

    return 0;
}

```

## 工具

介绍两个常用工具sysrepoctl和sysrepoconf

sysrepoctl可以添加、删除和修改YANG模型。sysrepo数据库只有一个，无论在程序中还是sysrepoctl操作的都是同一个sysrepo库，即通过程序中添加的YANG模型，sysrepoctl可以删除。

sysrepoctl常用选项：

- -l, --list 列出全部已经安全在sysrepo中的YANG模型。
- -i, --install 安装YANG模型
- -u, --uninstall 卸载安全的YANG模型

- -c, --change 修改YANG模型
- -U, --update 更新YANG模型

```
liuwei@47.93.188.23:mongoose $ sysrepoctl -l
Sysrepo repository: /home/liuwei/download/sysrepo-
master/build/repository
```

| Module Name          | Revision   | Flags | Startup Owner | Startup Perms | Running Perms | Submodules                | Features |
|----------------------|------------|-------|---------------|---------------|---------------|---------------------------|----------|
| example              | 2025-05-16 | I     | liuwei:liuwei | 600           |               |                           |          |
| ietf-datastores      | 2018-02-14 | I     | liuwei:liuwei | 444           |               |                           |          |
| ietf-factory-default | 2020-08-31 | I     | liuwei:liuwei | 600           |               | factory-default-datastore |          |

```
liuwei@47.93.188.23:mongoose $ sysrepoctl --uninstall example # 删除加
载到sysrepo中的example
```

sysrepocfg主要用于操作sysrepo数据库中的配置数据，支持对YANG模型实例化的数据进行增删改查和验证。

sysrepocfg常用选项：

- -d: 指定数据存储DB(running/startup/candidate)
- -m: 指定YANG模块名
- -X: 输出配置到文件或标准输出
- -I: 从文件或标准输入导入配置
- -E: 编辑当前数据存储内容
- -S: 设置xpath的值
- -G: 获取xpath的值

```
liuwei@47.93.188.23:~ $ sysrepocfg -d running -m example -X
<system xmlns="example">
  <items>
    <hostname>GE1_1</hostname>
    <ip-addresses>1.1.1.3</ip-addresses>
    <ip-addresses>2.2.2.2</ip-addresses>
  </items>
  <items>
    <hostname>GE2_1</hostname>
    <ip-addresses>1.1.1.3</ip-addresses>
    <ip-addresses>2.2.2.2</ip-addresses>
    <ip-addresses>3.3.3.3</ip-addresses>
    <ip-addresses>3.3.3.4</ip-addresses>
  </items>
  <items>
    <hostname>haha21</hostname>
    <ip-addresses>1.1.1.3</ip-addresses>
    <ip-addresses>2.2.2.2</ip-addresses>
  </items>
</system>
```

### 3、CONFIG DPDK

进程默认启动时，有CPU信息、大页内存以及PCIE内存通道两类信息需要提供给DPDK

- CPU信息。获取CPU配置信息



```
⇒ cat /etc/default/grub
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR="$(sed 's, release .*$,,g' /etc/system-release)"
GRUB_DEFAULT=saved
GRUB_DISABLE_SUBMENU=y
GRUB_TERMINAL="serial console"
GRUB_SERIAL_COMMAND="serial --speed=115200"
GRUB_CMDLINE_LINUX="default_hugepagesz=1G hugepagesz=1G hugepages=8
isolcpus=4,5,6,7 nohz_full=4,5,6,7 rcu_nocbs=4,5,6,7 numa=off
crashkernel=auto console=tty0 console=ttyS0,115200n8 iommu=off"
GRUB_DISABLE_RECOVERY="true"
GRUB_DISABLE_LINUX_UUID=true
```

- 大页内存

```
# 运行时预留大页
# 单NUMA
echo 1024 > /sys/kernel/mm/hugepages/hugepages-2048kB/nr_hugepages

# 多NUMA 分别设置(如下)或者使用上述方法平均分配
echo 1024 > /sys/devices/system/node/node0/hugepages/hugepages-
2048kB/nr_hugepages
echo 1024 > /sys/devices/system/node/node1/hugepages/hugepages-
2048kB/nr_hugepages

# dpdk-hugepages.py 管理大页的工具
# 某些内核版本可能不允许在运行时预留 1 GB 的 hugepages, 因此在启动时预留可能是唯一的选择
# mount挂载1GB大页

# 启动时预留大页
default_hugepagesz=1G hugepagesz=1G hugepages=4
```

- 支持NUMA架构

```
numa=on
```

#### ■ 内存通道

```
⇒ cat /etc/iftab
GE1.1 businfo 0000:06:00.3
GE1.2 businfo 0000:06:00.2
GE1.3 businfo 0000:06:00.1
GE1.4 businfo 0000:06:00.0
GE1.5 businfo 0000:05:00.3
GE1.6 businfo 0000:05:00.2
GE1.7 businfo 0000:05:00.1
GE1.8 businfo 0000:05:00.0
mgmt1 businfo 0000:0c:00.0
mgmt2 businfo 0000:0b:00.0

cat /sys/bus/pci/devices/0000:06:00.0/max_link_width
4
```

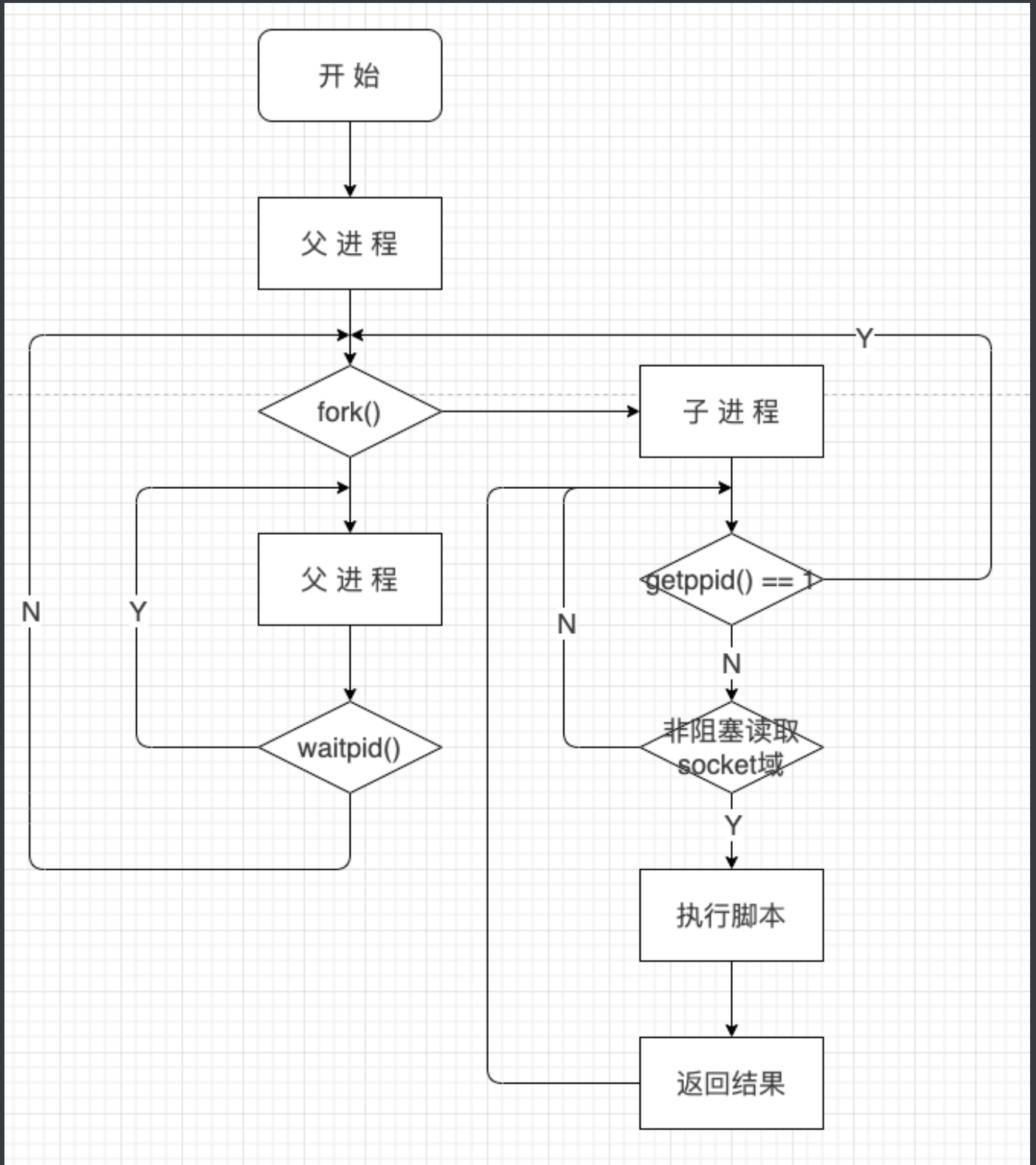
默认启动参数由配置决定。

## 4、父子进程脚本代理

脚本代理主要是为了处理包含DPDK进程不能执行脚本的问题。

父子进程脚本代理由crond启动，每分钟检查一次脚本代理是否存在，不存在则拉起。

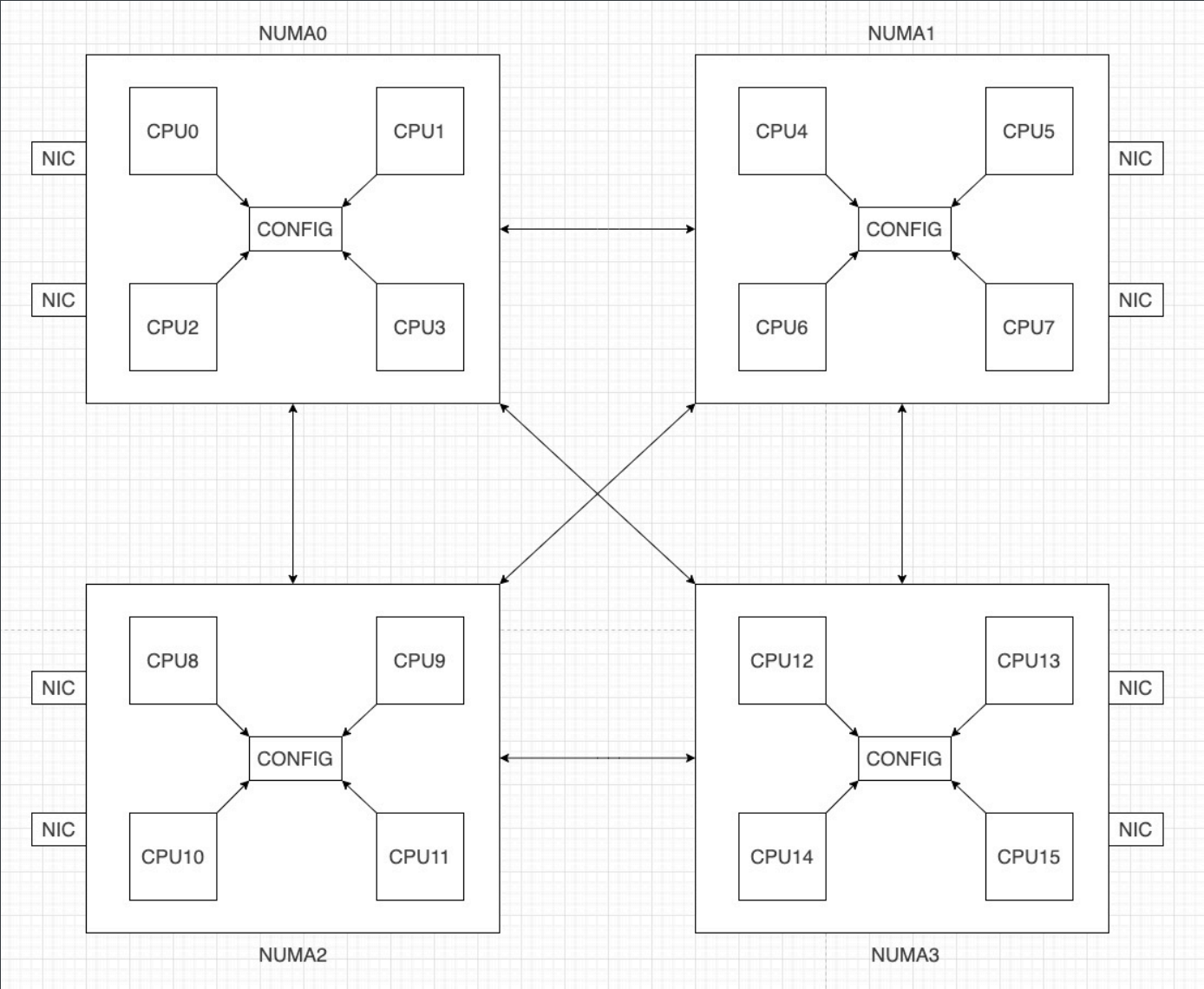
设计为父子进程则是为了让脚本执行更具有可靠性。父进程通过waitpid()接口监听子进程，一旦子进程退出，则父进程马上拉起子进程，子进程从socket域中读取任务并执行任务，周期性通过getppid()检查父进程是否存在，如果不存在则fork()子进程，自己变为父进程开始监听子进程，新子进程开始执行任务。具体流程如下所示：



### 三、APIGATEWAY

APIGATEWAY启动后，首先处理DPDK硬件配置以及NUMA相关的工作，处理完后启动API线程接收客户端配置，或者还有统计线程，最后启动DPDK线程。

DPDK线程编码中仅使用栈变量、线程私有变量和堆内存处理数据，以及线程启动时传入的NUMA初始化时创建的亲和自己的CPU句柄，每个DPDK线程根据这些配置句柄可以看到一份全量的配置，存储这些配置的内存都是亲和该CPU的，对DPDK线程而言，NUMA架构对它是透明的，编写它的代码时也不需要考虑NUMA架构，它在执行业务流程时仅访问启动时传入的句柄和非全局变量，如下所示配置访问架构：



## 1、main

```
int main(int argc, char *argv[])
{
    1、守护进程启动 daemon(0, 0);
    2、初始化EAL rte_eal_init(argc, argv);
    4、处理硬件信息：只获取信息不配置，配置下发的时候再配置。
        CPU管理
        网卡配置信息(NIC、MAC、RSS、DESCRIPTION、QUEUE等)
    5、初始化KNI
    6、初始化DPDK随机数种子
    7、为每个NUMA节点初始化MAC表、ARP表、ROUTE表、CT表以及HTTP/HTTPS等的大页内存池，把初始化句柄传入到每个线程
    8、启动API线程/统计线程等
    9、启动DPDK线程
}
```

## 2、API线程

### ■ API线程伪代码

阻塞获取客户端配置请求

```
switch (method) {
case POST: case PUT:
    执行配置检查
    配置复制到startup
```

```
if (为每个线程生成配置(或每个NUMA)，内存需要是目标线程亲和性的) {
    替换配置
    等待配置替换完成，释放旧配置，这里采用RCU替换机制，不用锁
    return 0;
} else {
    删除startup刚添加的配置
    return !0;
}
```

```
break;
```

```
case: DELETE:
    if (为每个线程生成删除配置后的内存) {
        替换配置
        等待配置替换完成，释放旧配置
        删除startup中的目标配置
        return 0;
    }

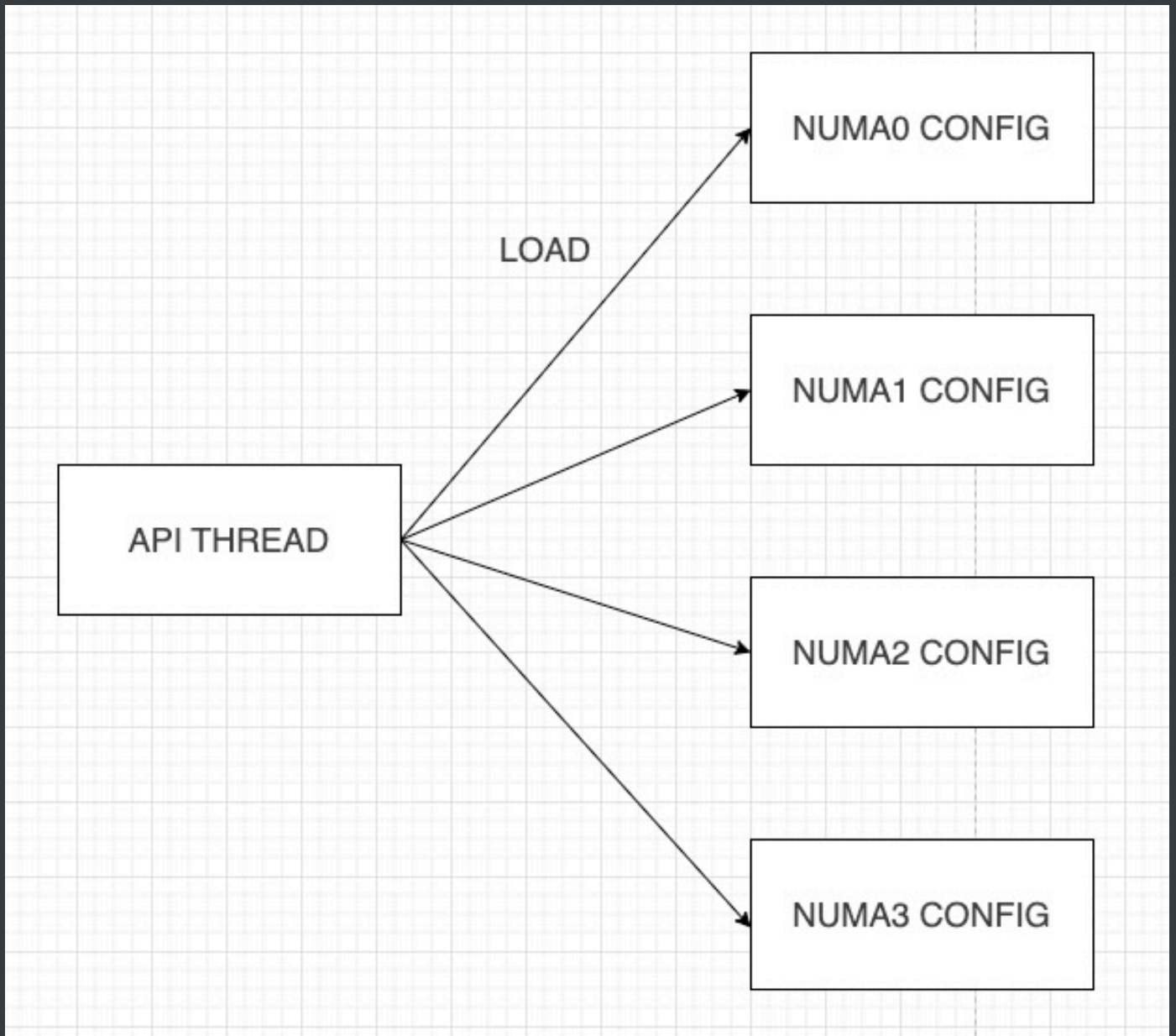
    return !0;

case GET:
    // 执行查询操作

default:
    // error;
}
```

#### ■ API线程操作配置示意图

API线程更新配置时对每个NUMA架构的句柄都要更新。二阶段更新方法来保证一致性，避免部分NUMA成功另一部分失败，第一步先检查配置，检查通过后，获取每个NUMA上足够的内存并初始化配置，都成功后，最后等待配置替换完成释放旧配置。



### 3、DPDK线程

DPDK线程：线程中不操作全局变量，只使用栈变量、线程私有变量、堆内存(NUMA内内存)以及配置句柄

```
void task_thread(void *arg)
{
    // 线程初始化(包含定时器)

    while (!quit) {
        for (int i = 0; i < 64; i++) {
            // 遍历网卡 从网卡中读取数据包
        }
    }
}
```

```

if (数据包个数不为0) {
    // 处理接收数据包
    // 处理发送数据包
}

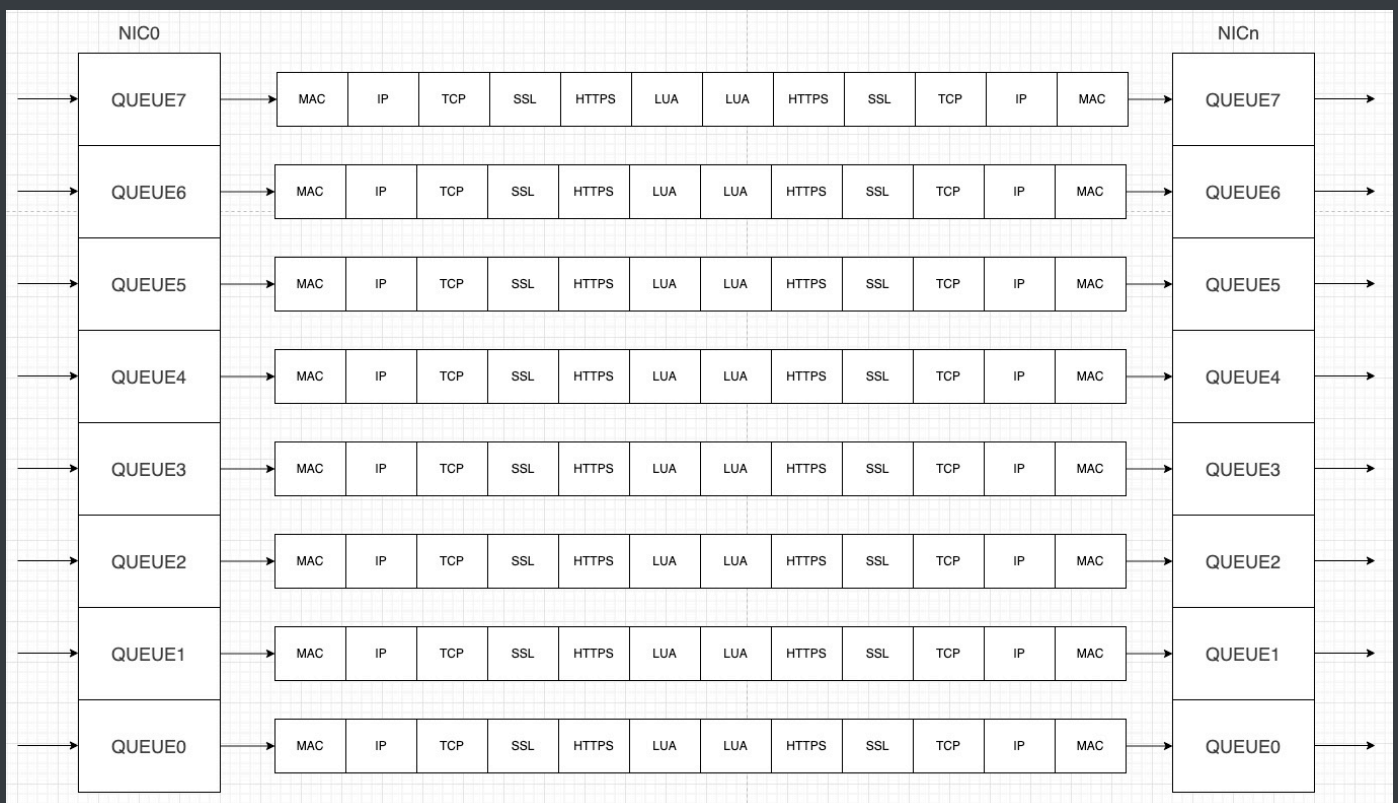
// 检查定时器
}

// RCU配置更新
}

// 释放初始化句柄
}

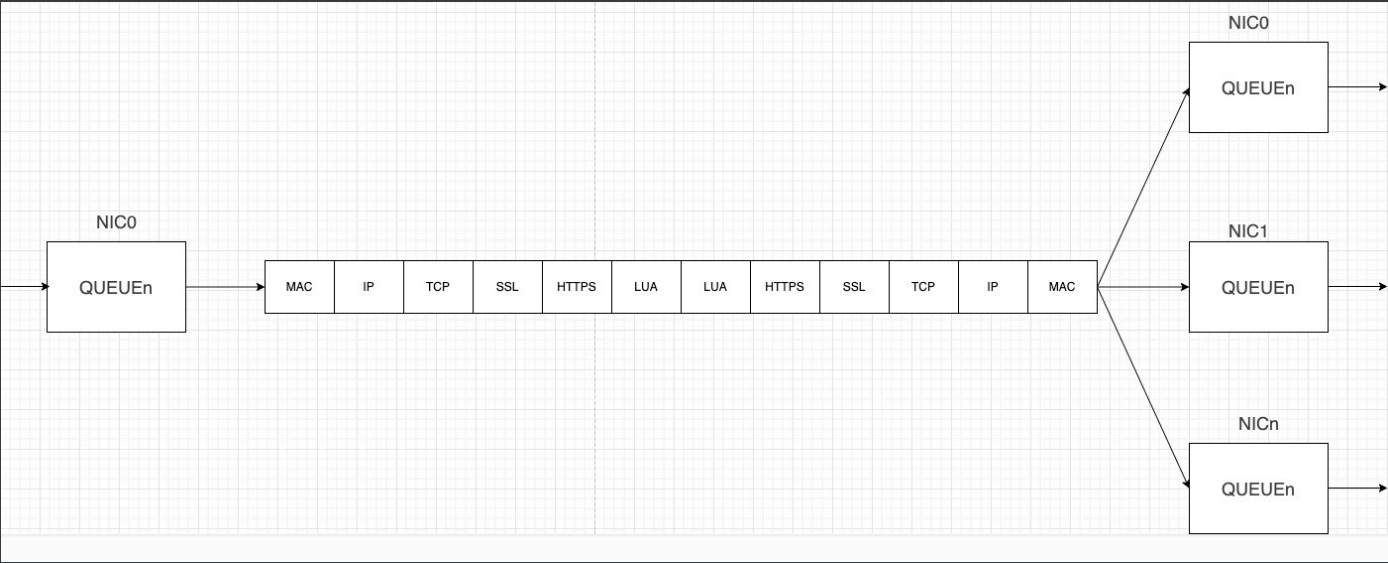
```

处理接收/发送数据包采用RUN TO COMPLETE模式，不再为各层划分线程，收到数据包后根据OSI模型，从底层往上处理，发送数据包时，从上往下，HTTP/HTTPS发送数据时获取一个符合要求的TCP的连接，即有效会话表，查询IP层路由信息，再然后查询ARP表到MAC层，最后根据MAC表获取网卡，从网卡的CPU的队列发送出去，一个工作线程处理全部逻辑，RUN TO COMPLETE分层示意图如下：





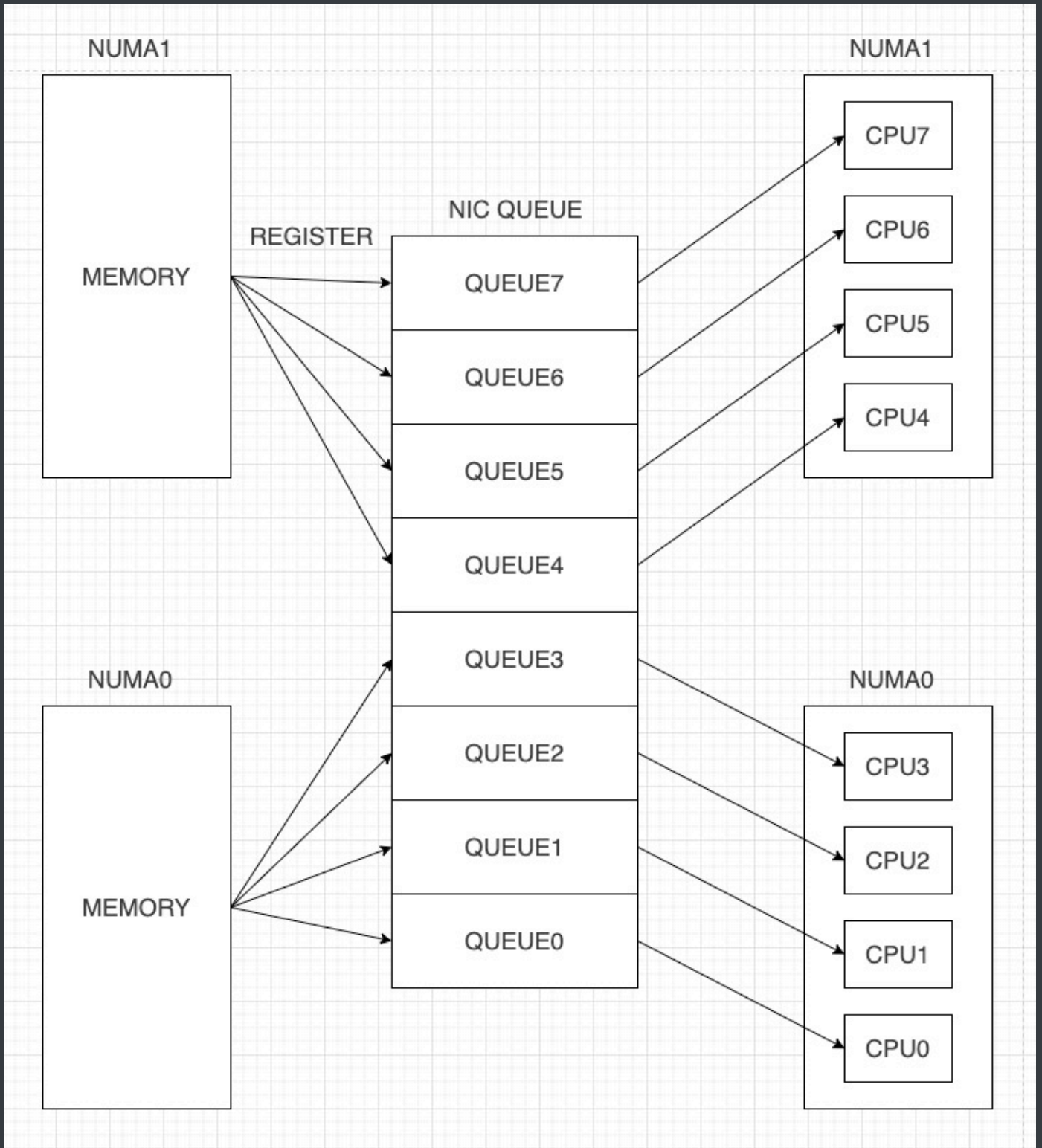
右边的网卡是可选的多个，队列编号固定，如下所示：



#### 4、网卡队列内存注册

每个网卡用CPU亲和的内存注册到网卡还是就近网卡NUMA的内存注册这个是很容易修改，后期测试下哪个性能高就行了。

如下是CPU亲和的内存注册以及CPU监听队列的情景



## 5、会话表设计

```
#ifndef CACHE_LINE
#define CACHE_LINE 64
#endif // CACHE_LINE

// 2 * CACHE
```

```

struct conn {
    struct list head; // nature order
    union nf_inet_addr src;
    union nf_inet_addr dst;
    uint16_t src_port;
    uint16_t dst_port;
    uint8_t protocol;
    uint8_t status;
    uint16_t dir : 1; // 0 - clientside 1 - serverside
    uint16_t reverse : 15;
    uint32_t seq;
    uint32_t ack_seq;
    struct recv_window {
        uint32_t next;
        uint32_t window;
        uint32_t irs; // init recv seq
    } recv_win;

    struct send_window {
        uint32_t unack;
        uint32_t next;
        uint32_t window;
        uint32_t wl1; // send window last update sequence
        uint32_t wl2; // send window last update acknowledgement
        uint32_t iss; // init send seq
    } send_win;

    void *recv_queue;
    void *send_queue;

    void *cb;
} __attribute__((aligned(CACHE_LINE)));

```

